

Technical Requirement Document (TRD) for Language Converter Platform

Aditya Thirani / Product Labs, IIIT Hyderabad

October 18, 2023

Contents

1	Introduction	2
2	System Overview	2
3	System Components	2
3.1	User Management System	2
3.2	Discourse Processor	2
3.3	Editable Grid System	2
3.4	USR Converter	2
3.5	Reviewer Assignment System	2
3.6	Database	2
3.7	User Interface	2
4	User Roles	2
5	Workflow	2
5.1	Admin Workflow	2
5.2	Annotator Workflow	3
5.3	Reviewer Workflow	3
6	Diagrams	3
6.1	Activity Diagram	3
6.2	Block Diagram	3
6.3	ER Diagram	3
6.4	Sequence Diagram	4
6.5	State Diagram	4
7	Backend Design	4
7.1	Programming Language: Python with Flask	4
7.2	Backend Components	4
7.2.1	User Management System	4
7.2.2	Discourse Processor	4
7.2.3	Editable Grid System	4
7.2.4	USR Converter	5
7.2.5	Reviewer Assignment System	5
7.3	Flask API Endpoints	5
7.4	Database: MySQL	5
7.5	MySQL Database Schema	5
8	Frontend Design: ReactJS	5
8.1	Frontend Architecture (React)	5
9	Project Timeline	6
10	Conclusion	6

1 Introduction

This document presents a detailed architectural overview of our Language Converter Platform, employing the Universal Semantic Representation (USR) framework. It is meticulously crafted to encapsulate and elucidate the pivotal architectural determinations foundational to the system's construction.

2 System Overview

The Language Converter Platform pioneers an innovative approach to linguistic translation by deconstructing discourses into elementary sentences, metamorphosing them into the USR, and subjecting these constructs to rigorous scrutiny and potential modification.

3 System Components

3.1 User Management System

Handles user roles such as Admin, Annotator, and Reviewer. This system will contain authentication and authorization modules.

3.2 Discourse Processor

A module dedicated to decomposing chapters into simplified sentences.

3.3 Editable Grid System

Presents the simplified sentences to the users for modifications.

3.4 USR Converter

Converts the modified sentences to Universal Semantic Representation.

3.5 Reviewer Assignment System

Assigns USR rows to reviewers for validation.

3.6 Database

Stores discourses, simplified sentences, USRs, and user/review data.

3.7 User Interface

A web interface developed using ReactJS, which interacts with backend services and the database.

4 User Roles

- **Admin:** Entrusted with the responsibility of uploading chapters and supervising the entire conversion process.
- **Annotator:** A pivotal actor in the simplification of sentences and is instrumental in generating the USR.
- **Reviewer:** Examines the derived USR with a critical lens and possesses the prerogative to edit it.

5 Workflow

5.1 Admin Workflow

The admin uploads a discourse, which is then processed into simplified sentences. The admin reviews these sentences in an editable grid and makes any required adjustments.

5.2 Annotator Workflow

The annotator annotates or modifies the simplified sentences in the editable grid. These sentences are then converted into USR.

5.3 Reviewer Workflow

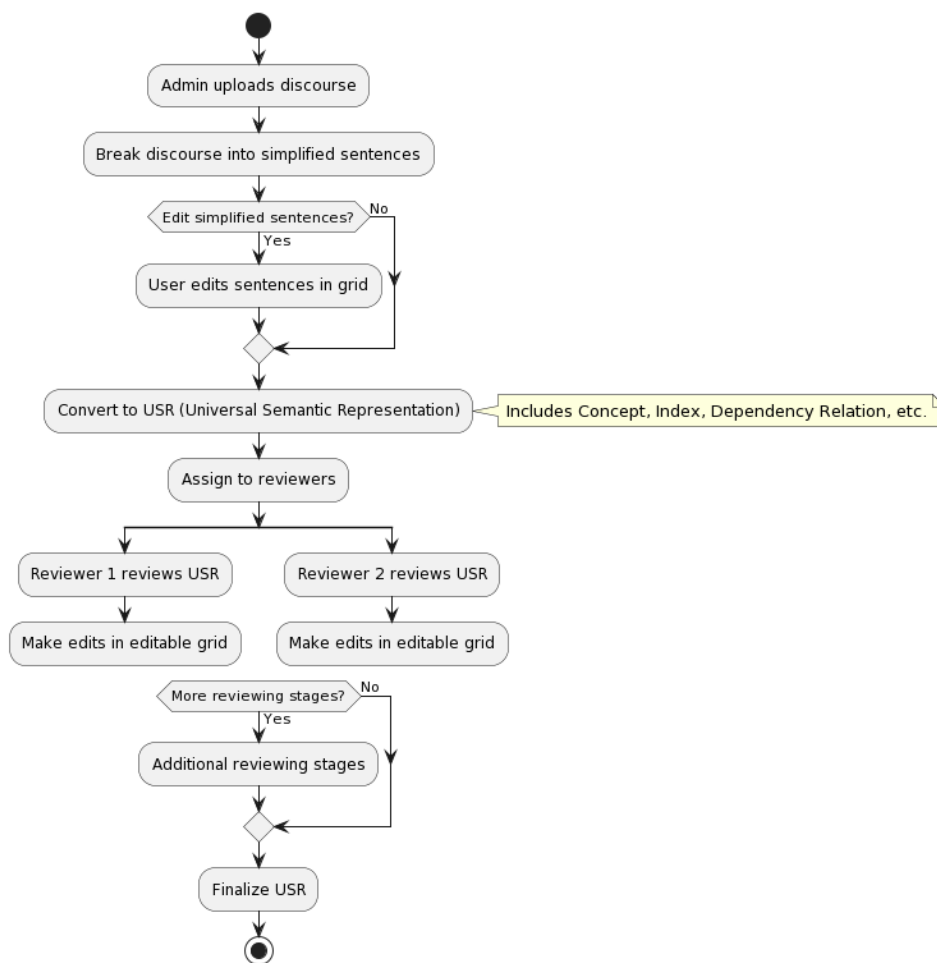
Reviewers are assigned USR rows for review. They are responsible for reviewing and validating USRs.

6 Diagrams

6.1 Activity Diagram

The Activity Diagram delineates the series of actions or flow of control in the system. It presents a bird's-eye view of the entire system operations. (See Figure 1)

Figure 1: Activity Diagram



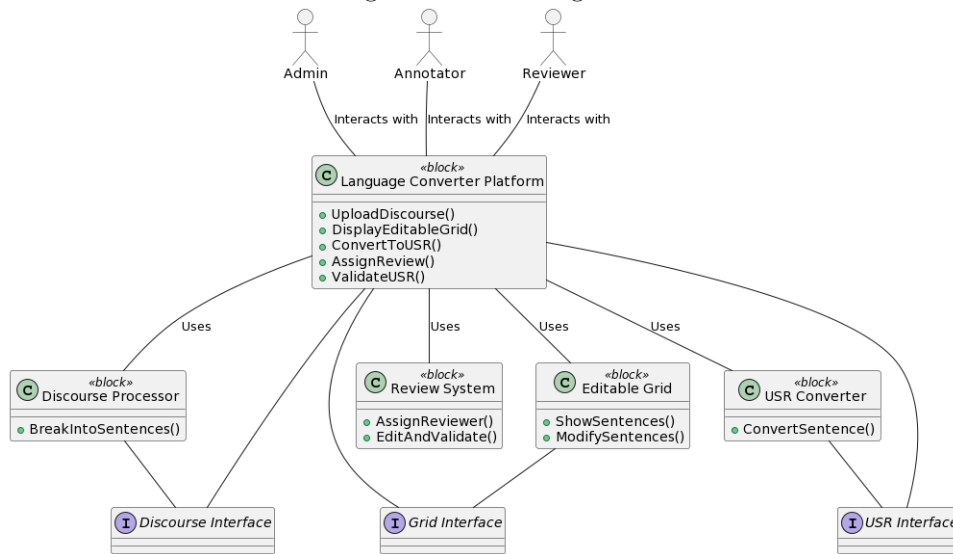
6.2 Block Diagram

The Block Diagram furnishes a high-level visualization of the system, elucidating how the primary components interact with each other.(See Figure 2)

6.3 ER Diagram

The ER (Entity Relationship) Diagram is instrumental in showcasing the logical structure of databases, detailing how entities are interconnected.(See Figure 3)

Figure 2: Block Diagram



6.4 Sequence Diagram

The Sequence Diagram portrays the interaction over time involving system objects. It captures the behavior of a single use case.(See Figure 4)

6.5 State Diagram

The State Diagram exhibits the sequence of states an object undergoes, influenced by events.(See Figure 5)

7 Backend Design

7.1 Programming Language: Python with Flask

Flask provides a lightweight and flexible framework, ideal for creating web applications. Leveraging Python means access to a vast ecosystem of libraries, simplifying and expediting development. Its scalability and ease of use make Flask an excellent choice for our backend operations.

7.2 Backend Components

Every component in the backend architecture serves a specific purpose:

7.2.1 User Management System

Given the variety of users and roles (Admin, Annotator, and Reviewer), there's a need for a robust system that ensures secure authentication and role-specific authorization.

7.2.2 Discourse Processor

For the platform to effectively convert language, breaking down complex chapters into simpler sentences is crucial. This granularity facilitates easier annotation and subsequent conversion.

7.2.3 Editable Grid System

An interactive UI is crucial for users to make necessary modifications. A grid system presents the information neatly and enables efficient editing.

7.2.4 USR Converter

The crux of the platform, this component's role is to convert simplified sentences into a standardized format (USR) which can be easily interpreted and reviewed.

7.2.5 Reviewer Assignment System

To ensure the quality and accuracy of USRs, a reviewer system is indispensable. This component ensures that USRs are vetted by experts before they are finalized.

7.3 Flask API Endpoints

- `/login`: Authenticate users.
- `/upload-discourse`: Admin uploads discourse content.
- `/get-simplified-sentences`: Retrieve simplified sentences.
- `/update-sentence`: Update sentence post-editing.
- `/convert-to-USR`: Conversion of sentence to USR.
- `/assign-review`: Assign USR rows to specific reviewers.
- `/review-USR`: Submit USR reviews.

7.4 Database: MySQL

MySQL, a robust and reliable relational database offers high performance, scalability, and is backed by a strong community. It is known for its quick processing, proven reliability, ease of use, and robust nature, especially for web-based applications. Being ACID-compliant ensures reliable processing, and its open-source nature gives us flexibility in terms of modifications and customizations. Furthermore, MySQL integrates seamlessly with Python, allowing for a more efficient data handling process.

7.5 MySQL Database Schema

- `users`: Storing user details with roles like admin, annotator, and reviewer.
- `discourses`: Storing uploaded discourses.
- `simplified_sentences`: Storing the derived sentences from discourses.
- `usr_entries`: Storing USRs with fields representing concepts, indices, etc.
- `reviews`: Storing reviews for each USR.

8 Frontend Design: ReactJS

ReactJS is chosen for its efficiency and flexibility. It allows developers to create large web applications that use data which can change over time, without reloading the page. Its main goal is to be fast, simple, and scalable. React components can easily be reused across the application, reducing the development time and ensuring consistency in the UI. Moreover, its virtual DOM implementation and various rendering optimizations make it incredibly fast and efficient.

8.1 Frontend Architecture (React)

- **Login Component**: Facilitating user sign-ins.
- **Dashboard Component**:
 - **Dashboard Component**: Provides functionalities depending on user roles.
 - **Upload Component**: Enables the admin to upload discourses.
 - **EditableGrid Component**: Displays sentences for editing and modification.
 - **USRReview Component**: Allows reviewers to view and validate USRs.

9 Project Timeline

Here is the Gantt chart outlining the project timeline for the Language Converter Platform. (See figure 6)

10 Conclusion

This TRD is a comprehensive representation of the Language Converter Platform's architecture and design principles. The tools and frameworks have been chosen based on their efficacy, robustness, and compatibility with our project requirements. As development progresses, this document may undergo revisions to stay aligned with any emergent architectural or design modifications. It serves as a beacon guiding our development endeavors, ensuring the system aligns with its envisioned goals and objectives.

Figure 3: ER Diagram

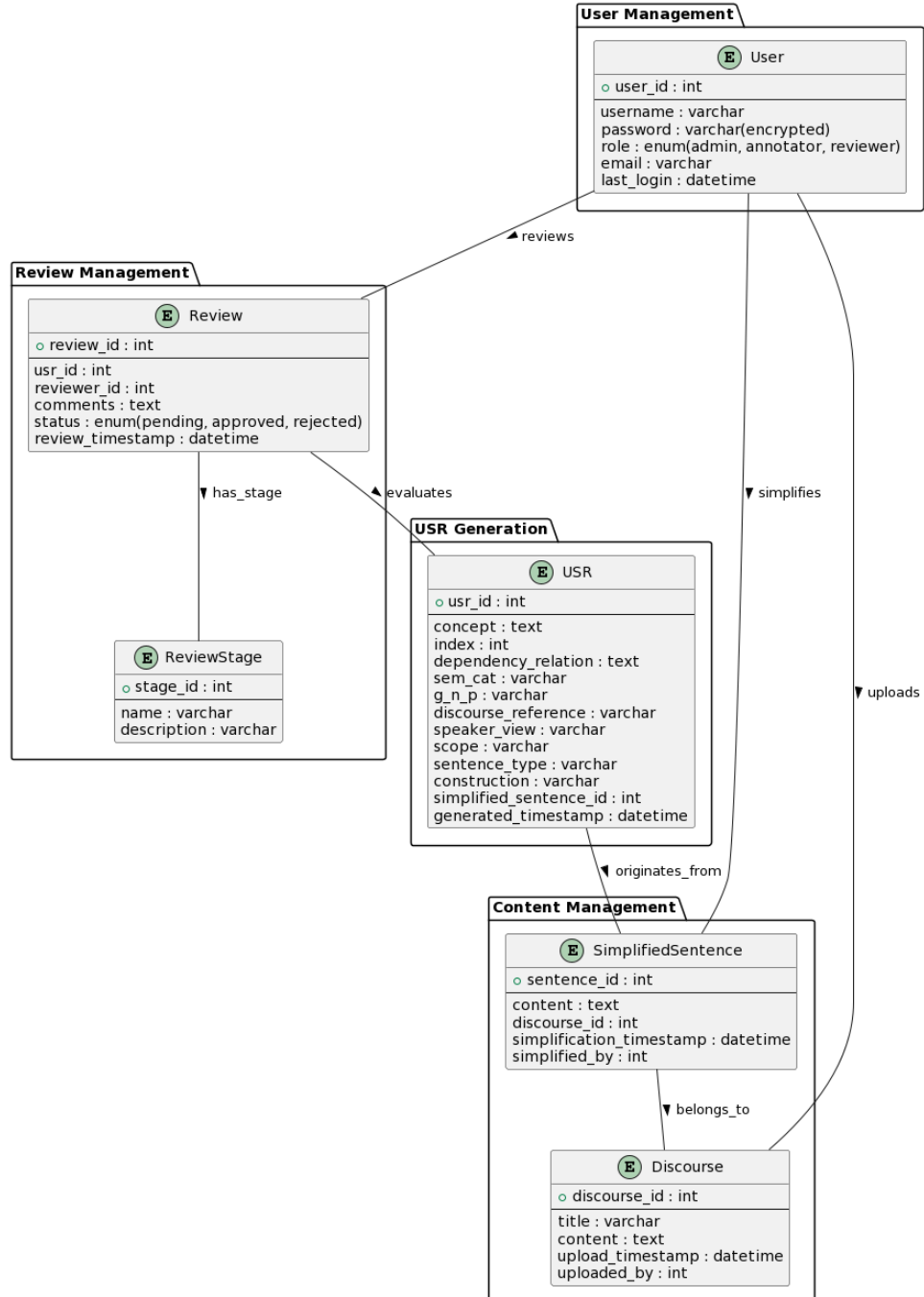


Figure 4: Sequence Diagram

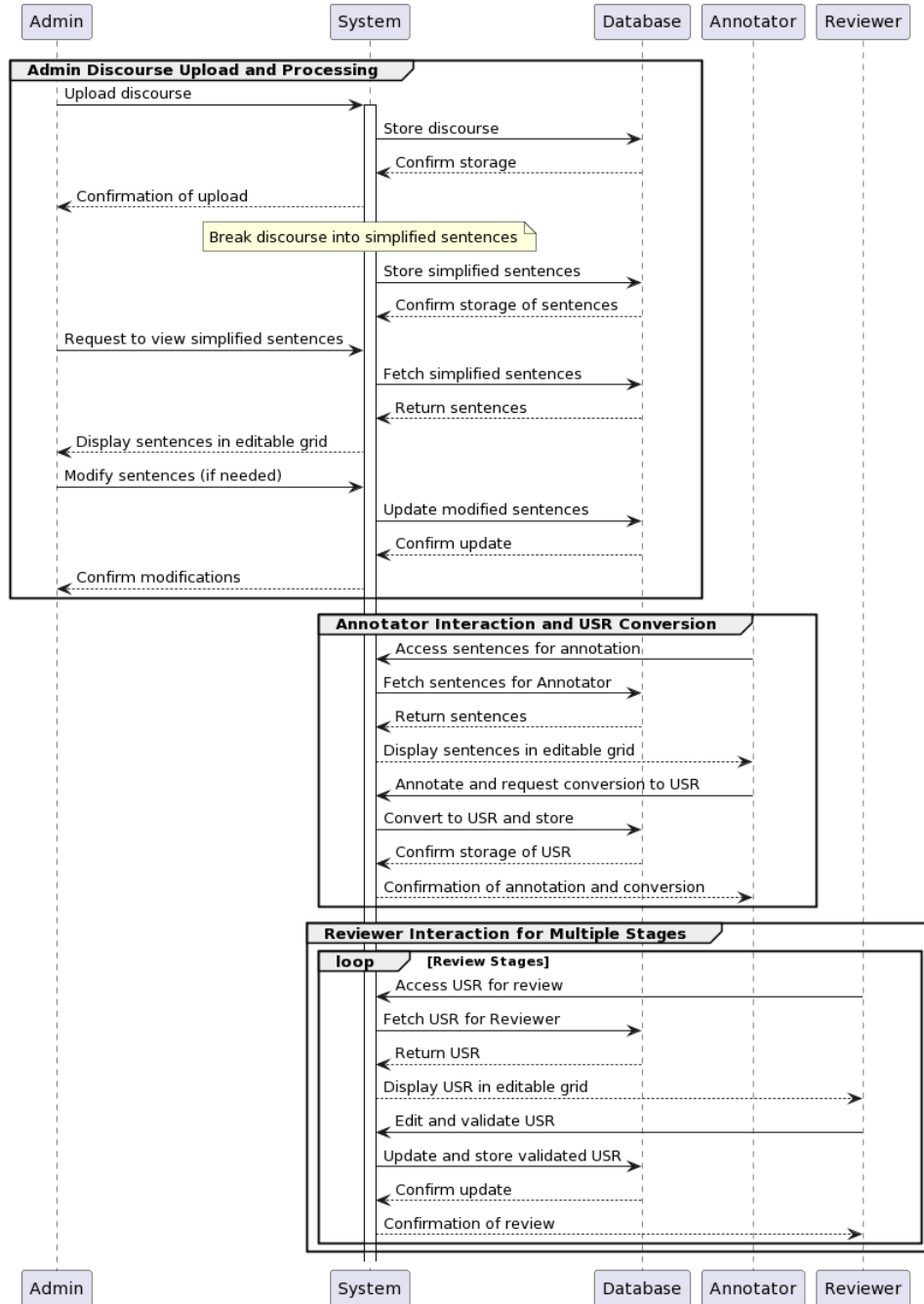
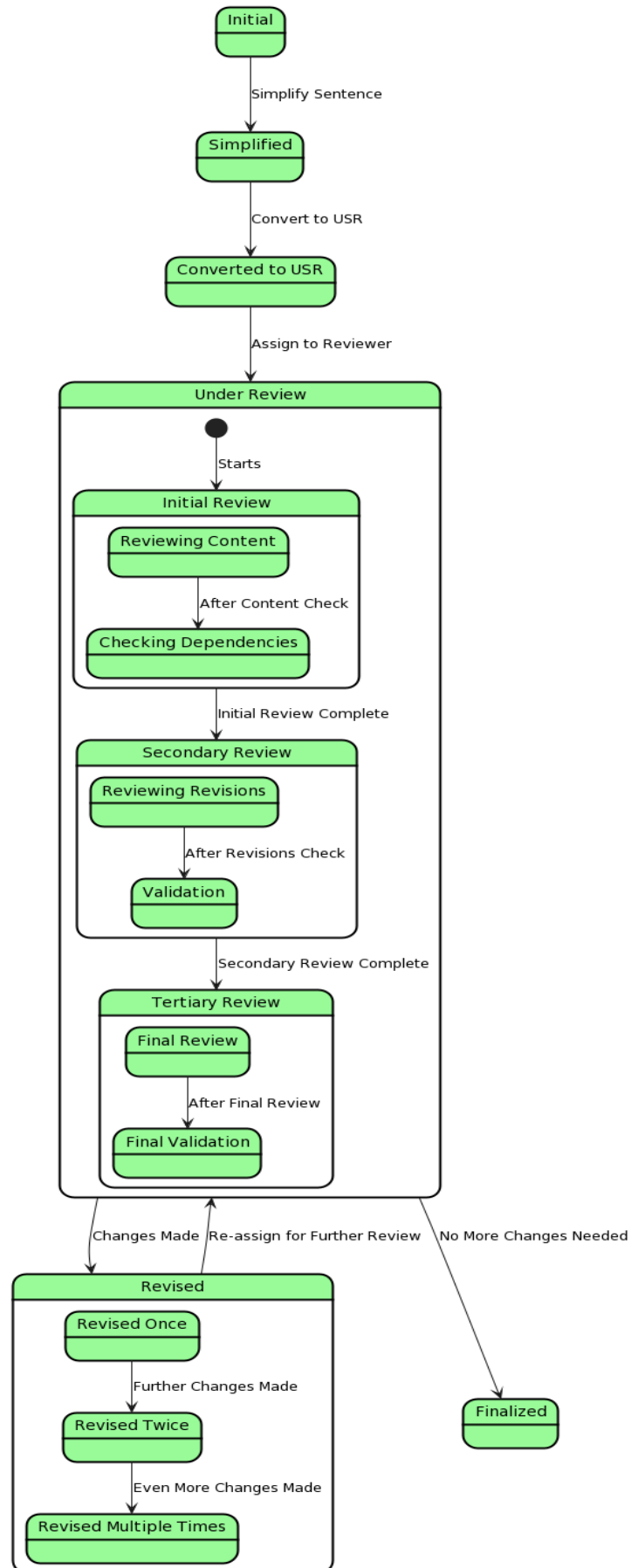


Figure 5: State Diagram



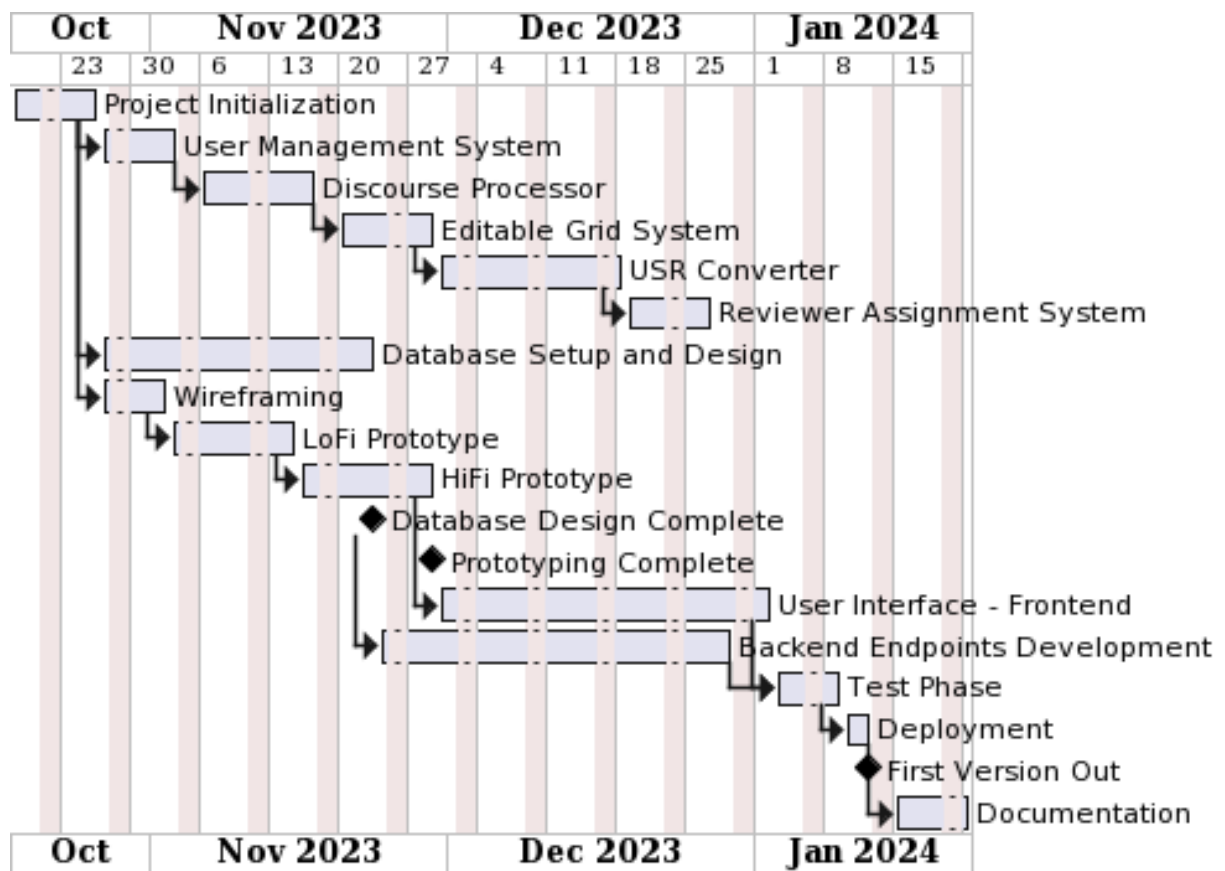


Figure 6: Gantt Chart for Language Converter Platform